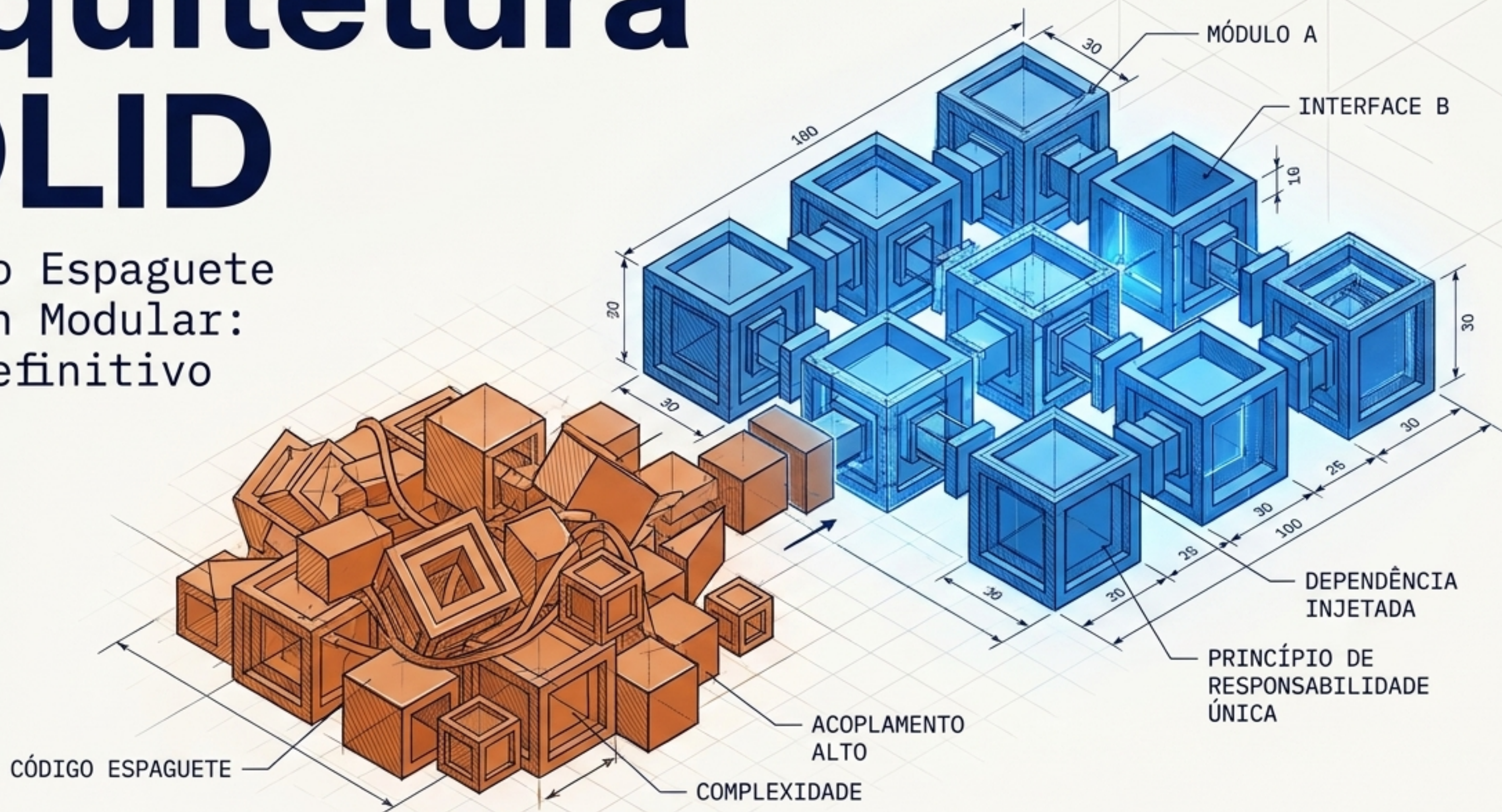
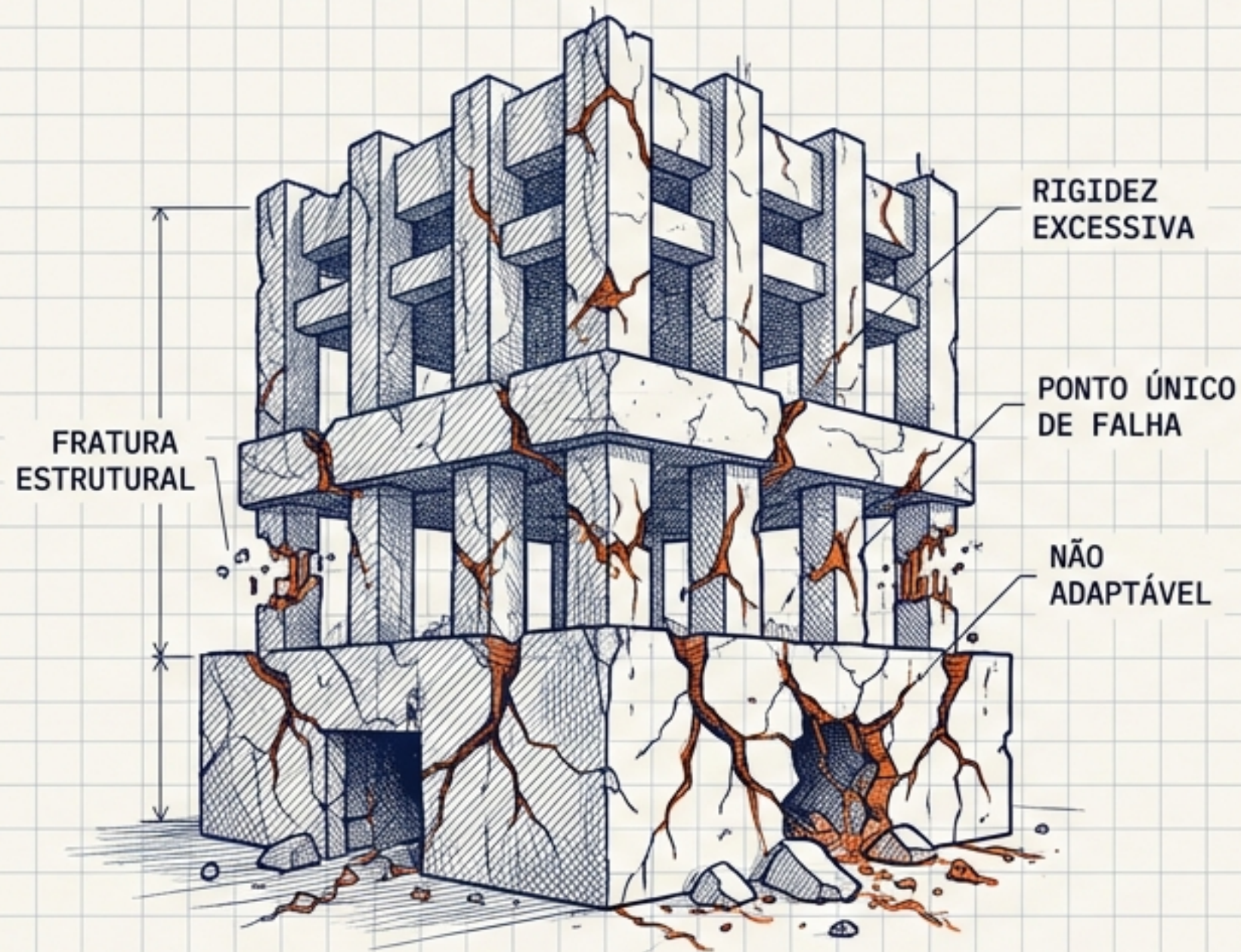


Arquitetura SOLID

Do Código Espaguete
ao Design Modular:
O Guia Definitivo

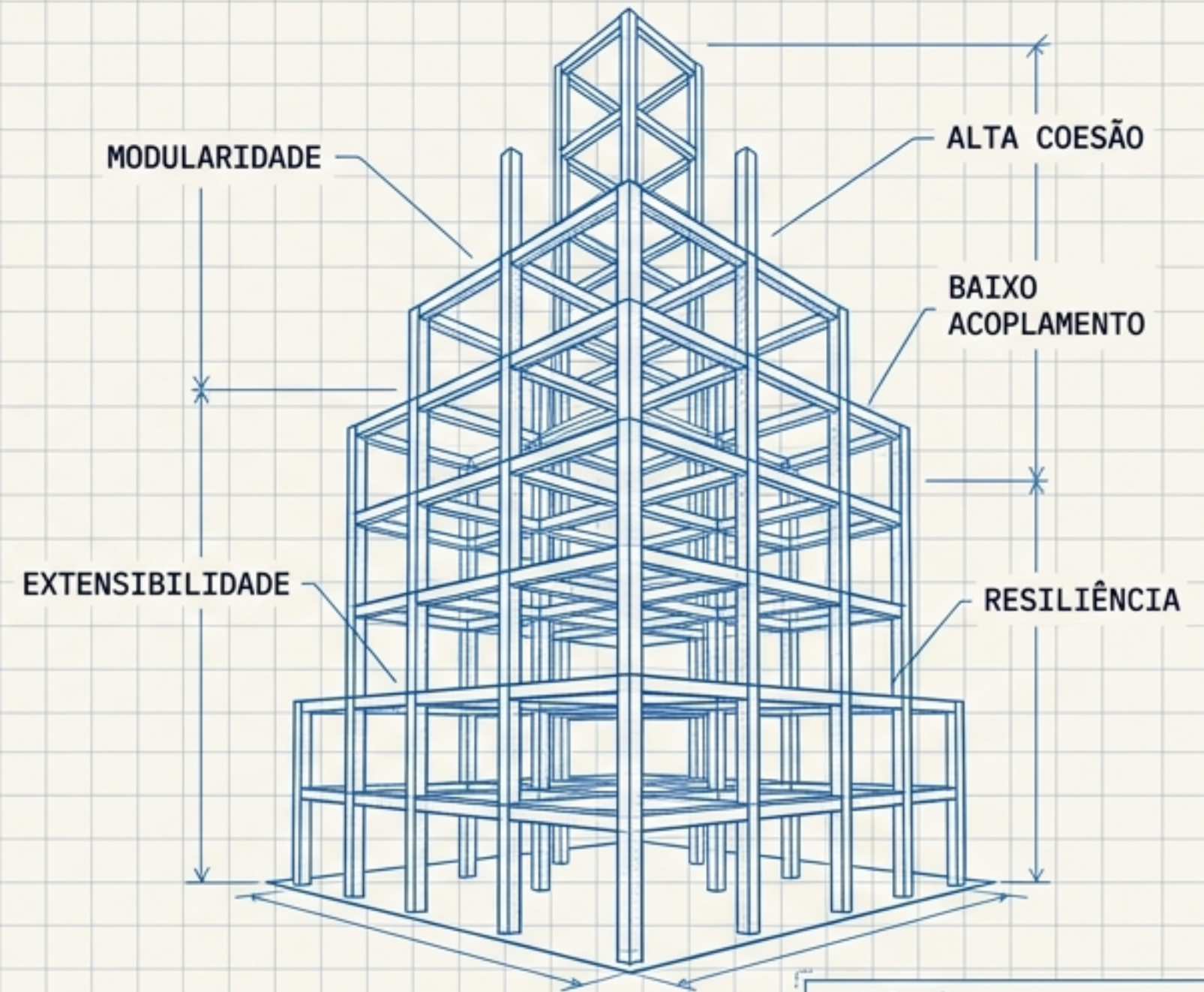


O Fim das Reescritas Constantes



O Problema:
Código rígido, frágil e não reutilizável.

Criados no início dos anos 2000 por Robert C. Martin (Uncle Bob), os princípios SOLID garantem que o software mude de requisitos sem precisar ser reescrito do zero.

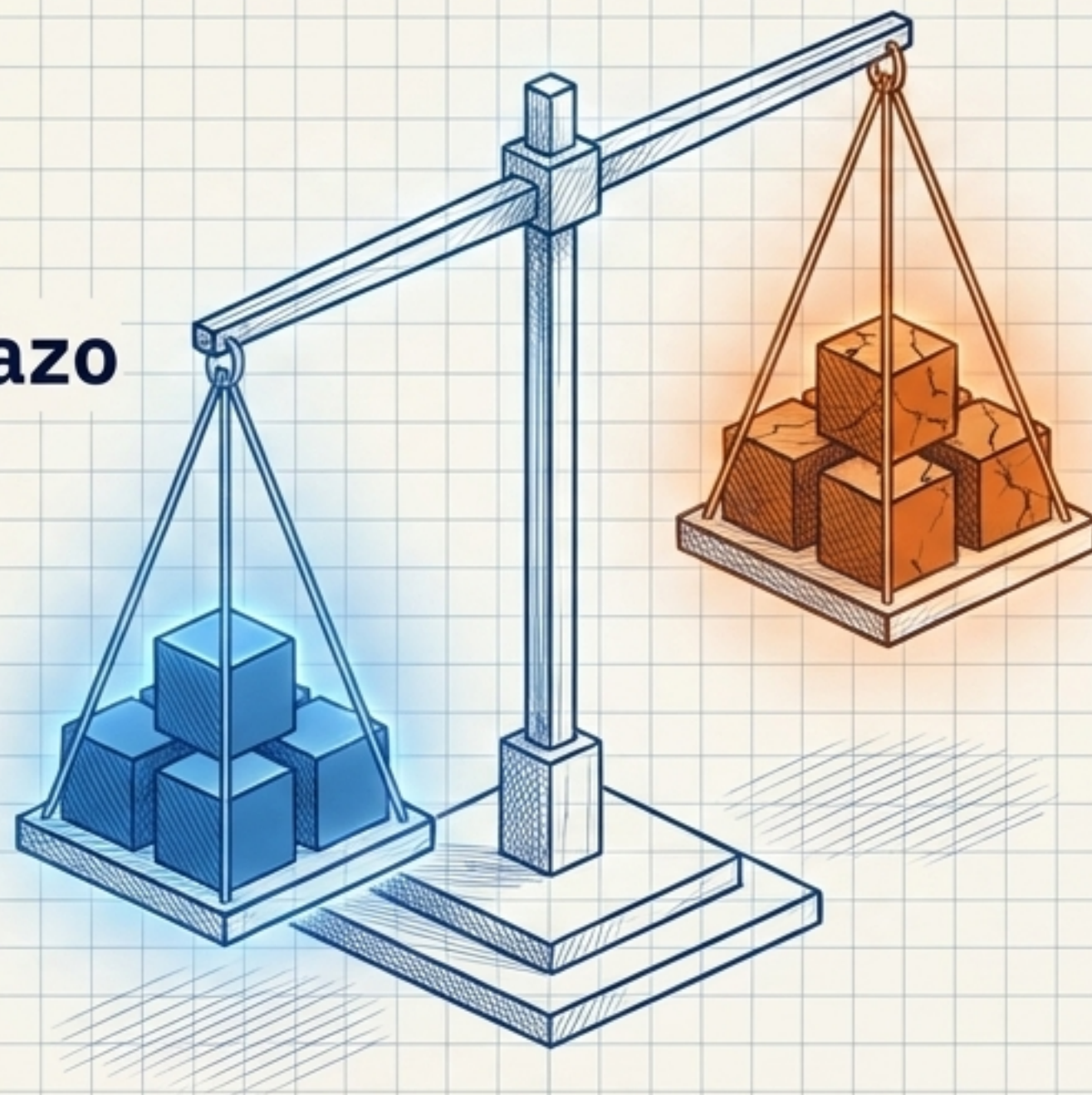


A Solução:
Estruturas que resistem à evolução do tempo.

A Balança da Arquitetura: Não Existe Bala de Prata

Ganhos a Longo Prazo

- Manutenção Facilitada
- Reutilização de Código
- Alta Testabilidade
- Flexibilidade e Escalabilidade



Custos Iniciais

- Maior Complexidade de Design
- Sobrecarga (Overhead) de Estrutura
- Maior Volume de Classes e Interfaces

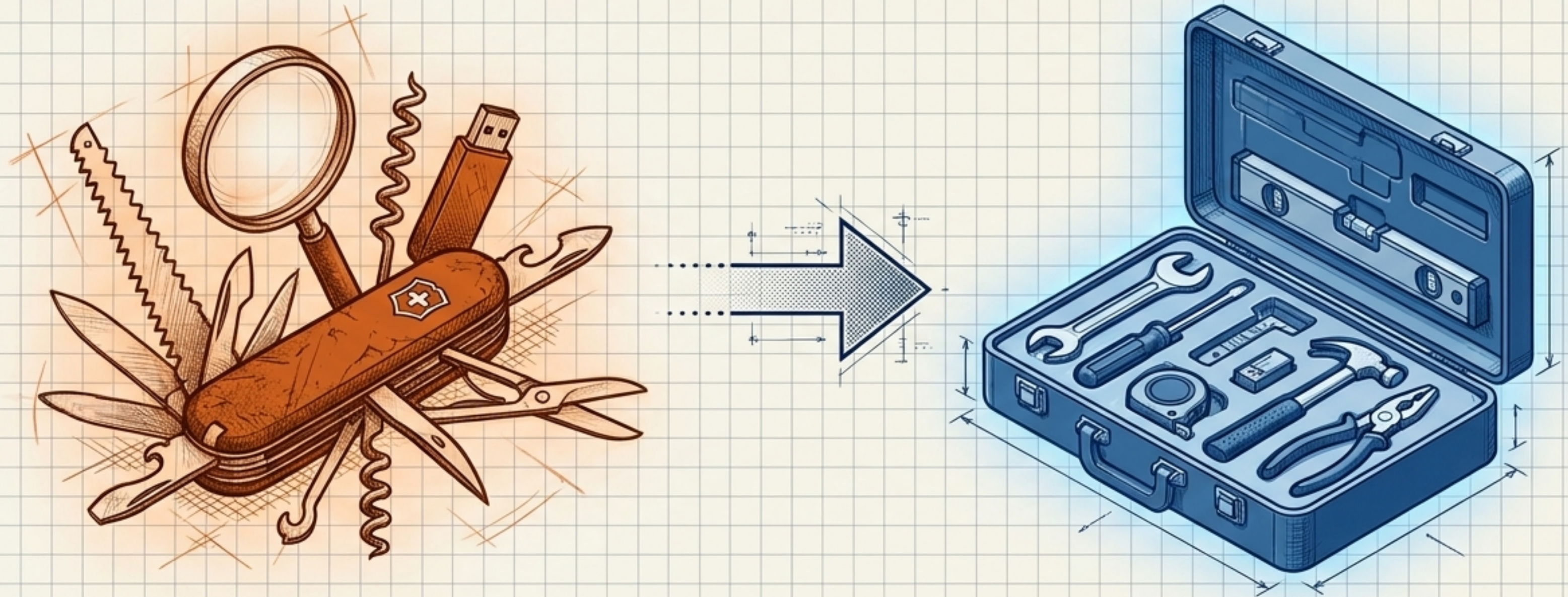
Implementar SOLID exige planejamento. O overhead inicial é amplamente superado pela manutenibilidade ao longo da vida útil do sistema.

O Diagnóstico SOLID

Princípio	A Regra	O Sintoma (Mau Cheiro)	A Cura
S (SRP)	Uma única razão para mudar.	Classes com 'E' no nome; métodos fazendo cálculo e relatório.	Extraí classes por responsabilidade.
O (OCP)	Aberto para extensão, fechado para modificação.	Cascatas infinitas de if/else ao adicionar novos tipos.	Usar interfaces e polimorfismo.
L (LSP)	Subclasses não quebram o contrato da base.	Sobrescrever método para permitir comportamento oposto (ex: saldo negativo).	Refatorar a hierarquia de herança.
I (ISP)	Interfaces pequenas e específicas.	Forçado a implementar métodos vazios ou lançar exceções.	Segregar interfaces obesas em menores.
D (DIP)	Depender de abstrações, não de implementações.	Instanciar classes concretas direto na lógica de negócios.	Injeção de dependência.

Single Responsibility Principle (SRP)

A Regra: Cada classe deve ter apenas uma razão para mudar.



Quando uma classe tem múltiplas responsabilidades, mudanças em uma parte introduzem bugs em outra. O acoplamento aumenta e a testabilidade desaparece.

0 Desmembramento do Objeto Deus

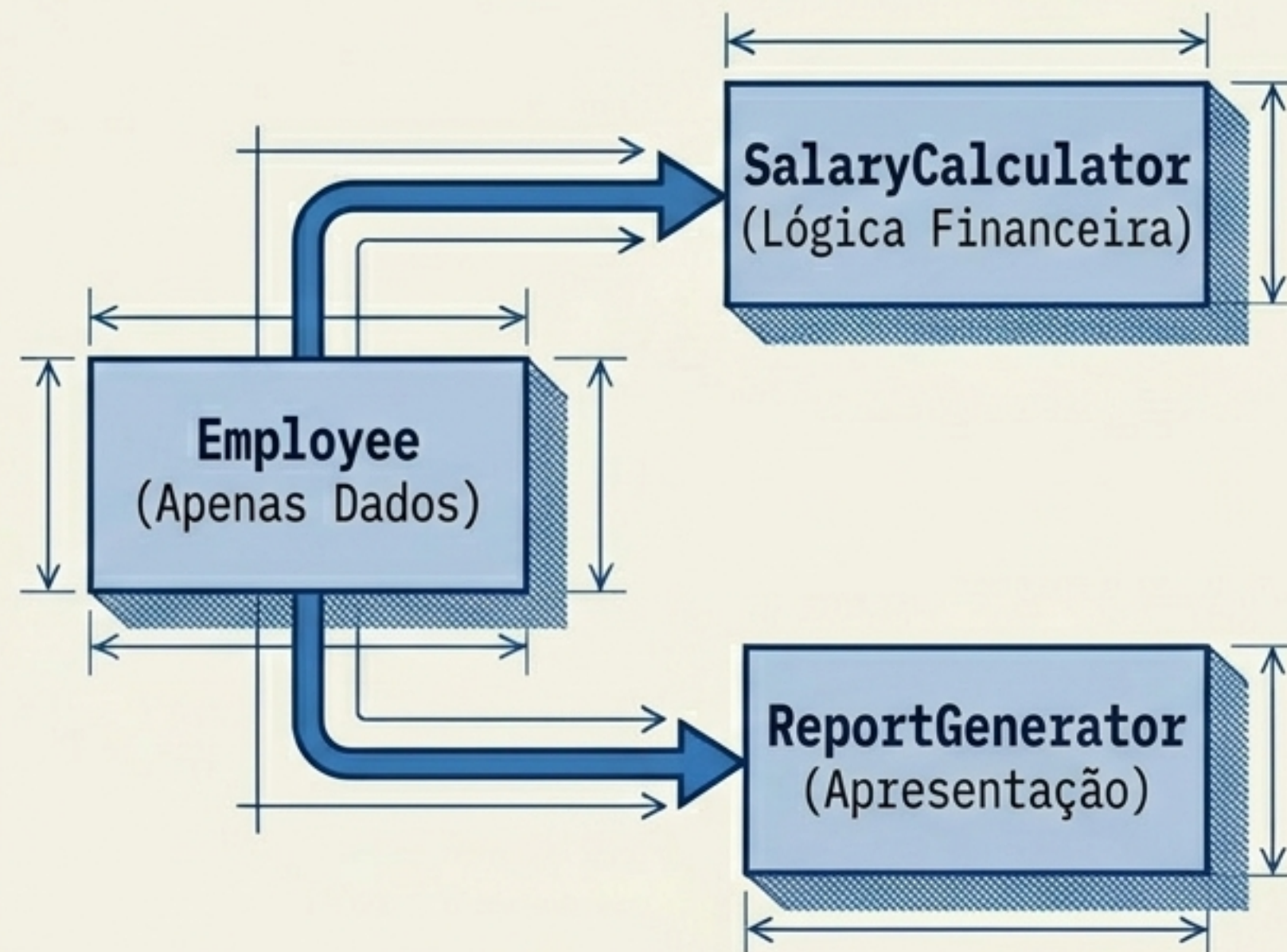
Antes: Múltiplas Razões para Mudar

```
class Employee {  
    String name;  
    double baseSalary;
```

```
    double calculateSalary() { ... }  
    String generateReport() { ... }
```

```
}
```

Depois: Responsabilidades Segregadas



Agora, alterar o formato do relatório não arrisca quebrar o cálculo do salário.
A coesão aumenta.

Open/Closed Principle (OCP)

A Regra: Entidades devem estar abertas para extensão, mas fechadas para modificação.



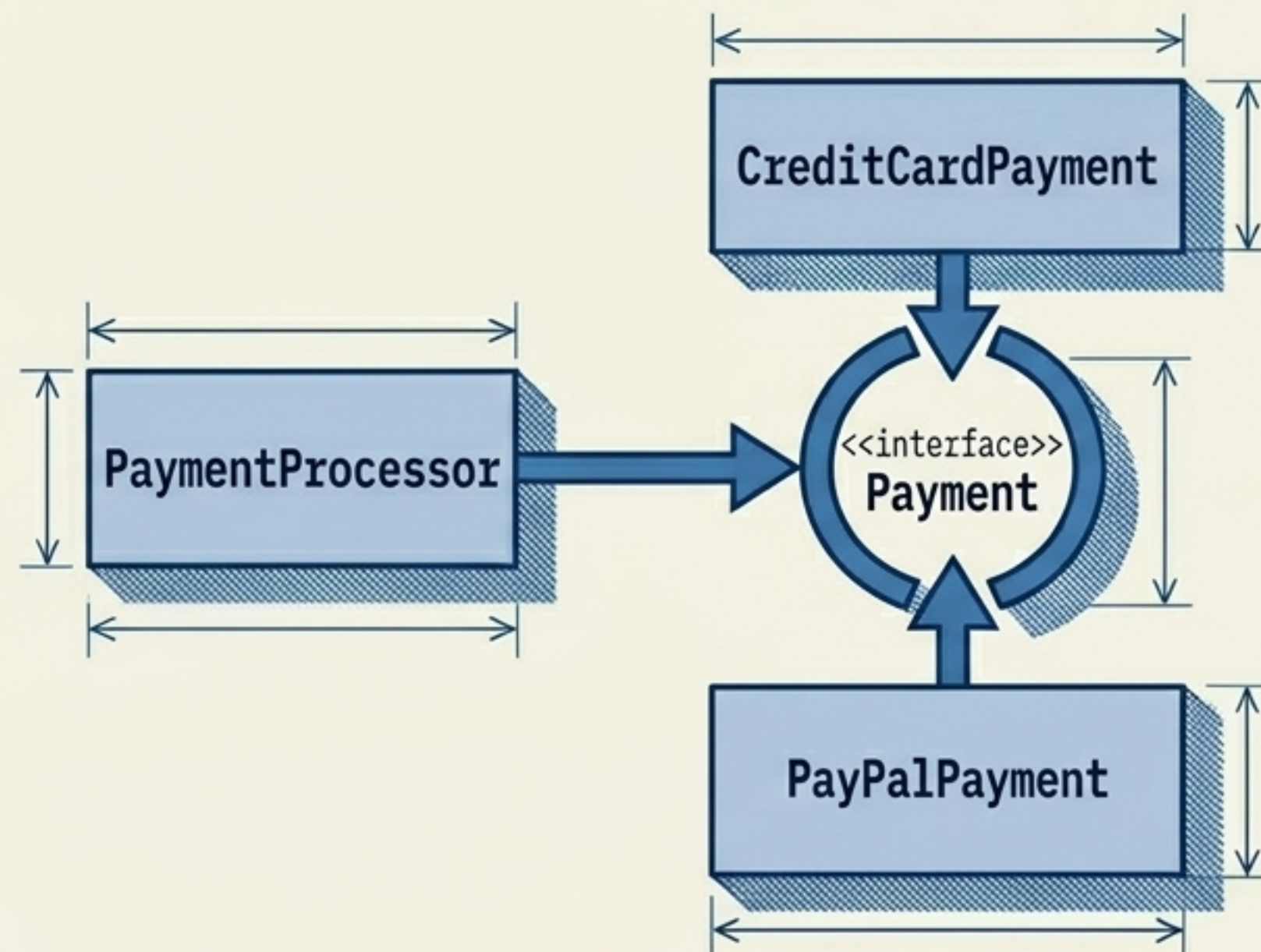
O comportamento de um módulo deve poder ser estendido criando novos códigos baseados em abstrações, sem a necessidade de tocar no código-fonte original.

Erradicando a Cascata de Condicionais

A Violação: Código Rígido

```
class PaymentProcessor {  
  void process(String type) {  
    if (type.equals('creditCard')) { ... }  
    else if (type.equals('paypal')) { ... }  
    // Risco: Todo novo método exige  
    alterar aqui  
  }  
}
```

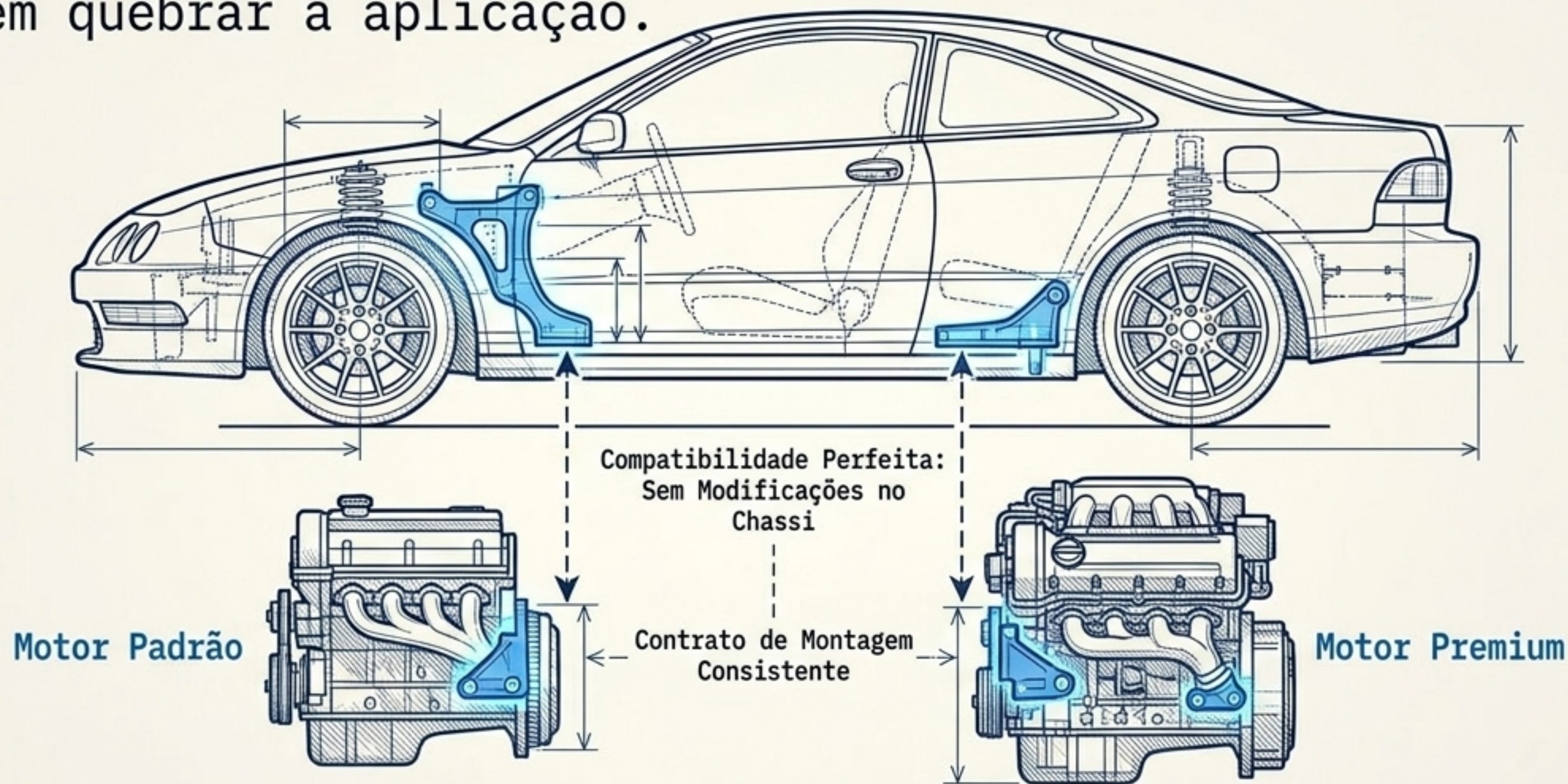
A Cura: Polimorfismo



Novos métodos de pagamento são adicionados criando uma nova classe.
Tocamos em zero linhas do código existente.

Liskov Substitution Principle (LSP)

A Regra: Classes derivadas devem poder substituir suas classes base sem quebrar a aplicação.



A subclasse deve manter o "contrato" da classe base. Ela não pode introduzir comportamentos inesperados ou forçar o sistema a tratar exceções.

Mantendo a Promessa da Herança

0 Risco: Quebra de Expectativa

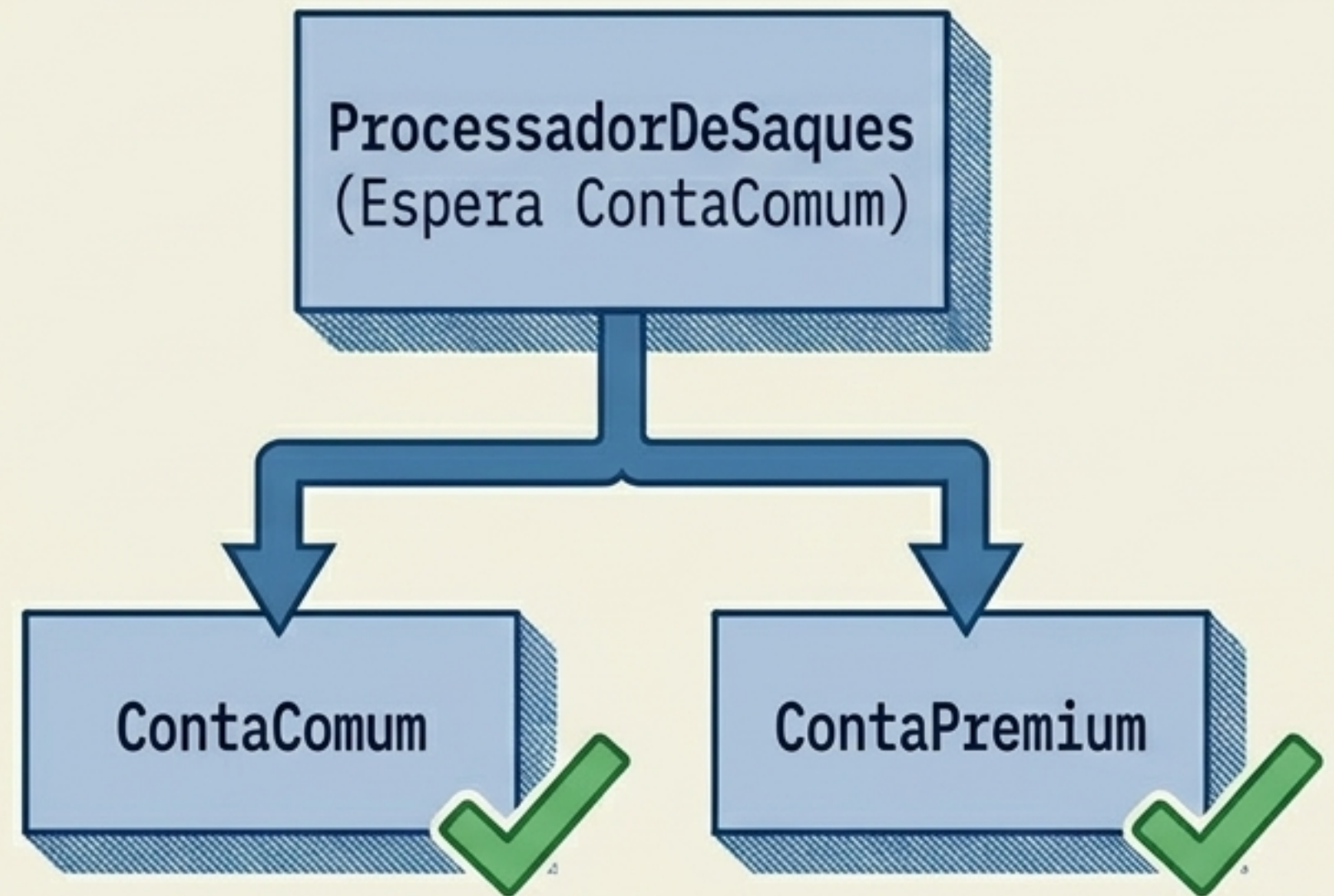
```
class ContaPremium extends
  ContaComum {
  @Override
  public void sacar(double valor) {
    saldo -= valor; // Permite saldo
    negativo
  }
}
```



0 sistema espera que uma ContaComum
nunca fique negativa.

Se uma função espera a classe pai, a classe filha não pode
sabotar essa função silenciosamente.

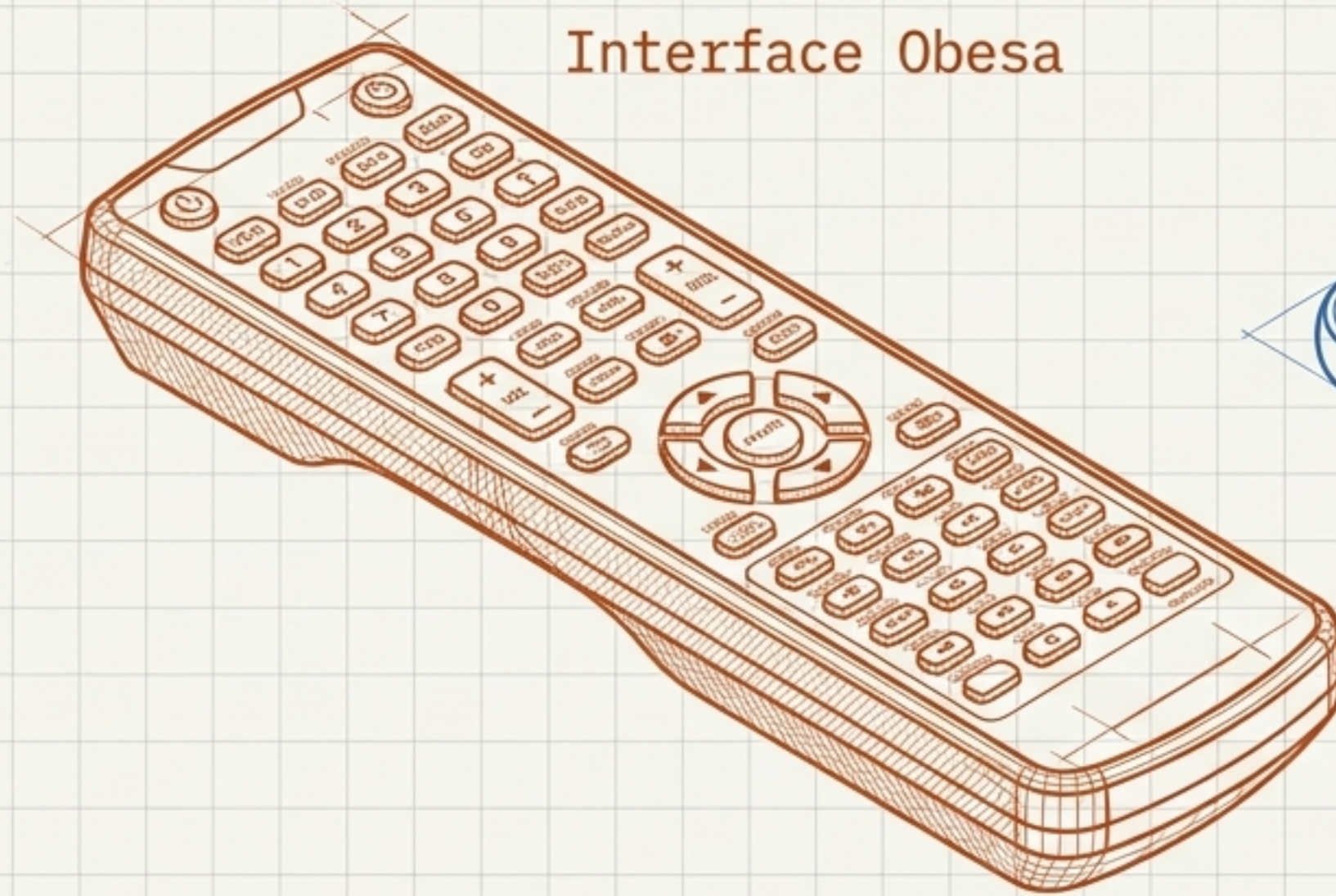
0 Contrato Garantido



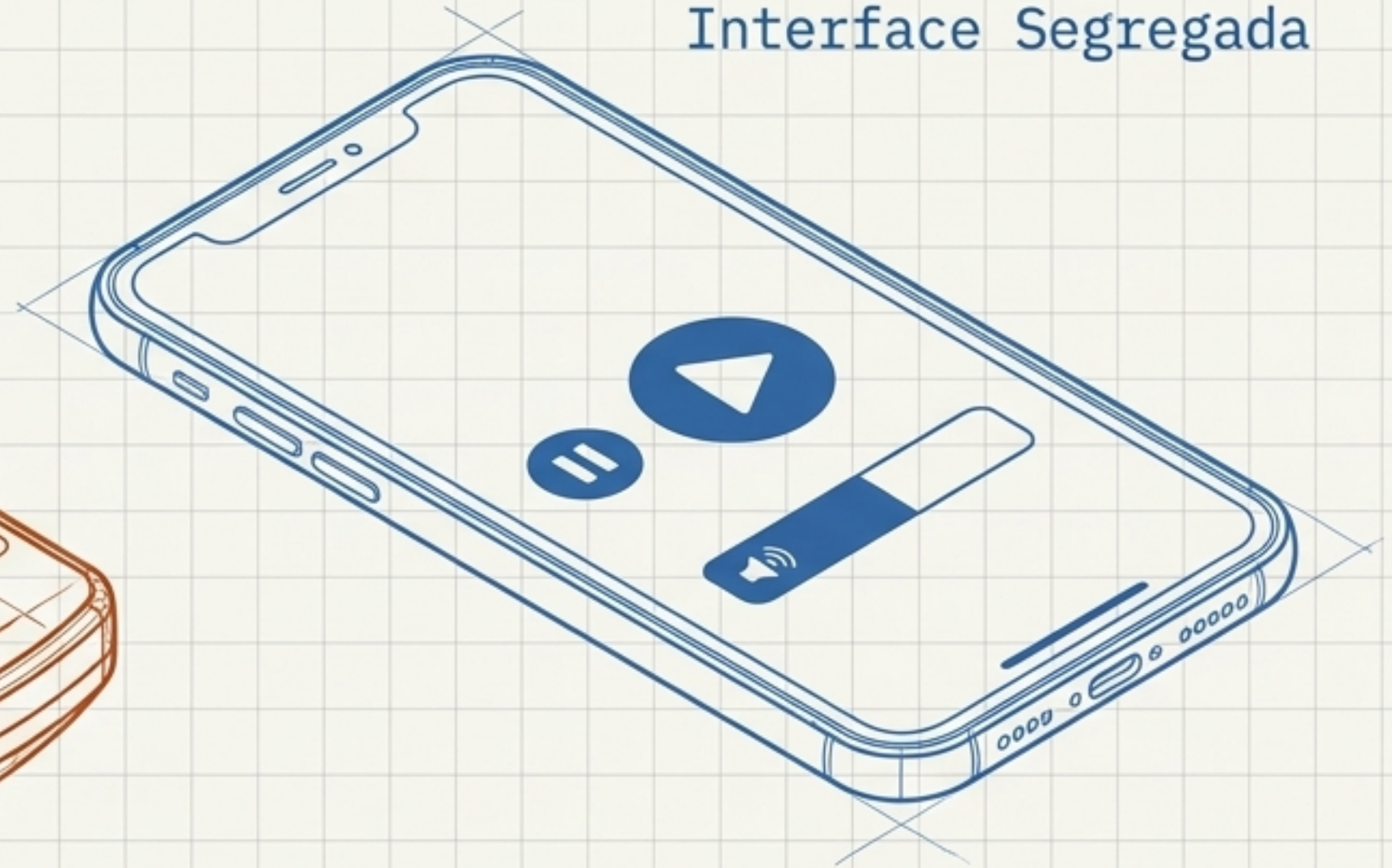
Interface Segregation Principle (ISP)

A Regra: Nenhum cliente deve ser forçado a depender de métodos que não utiliza.

Interface Obesa



Interface Segregada



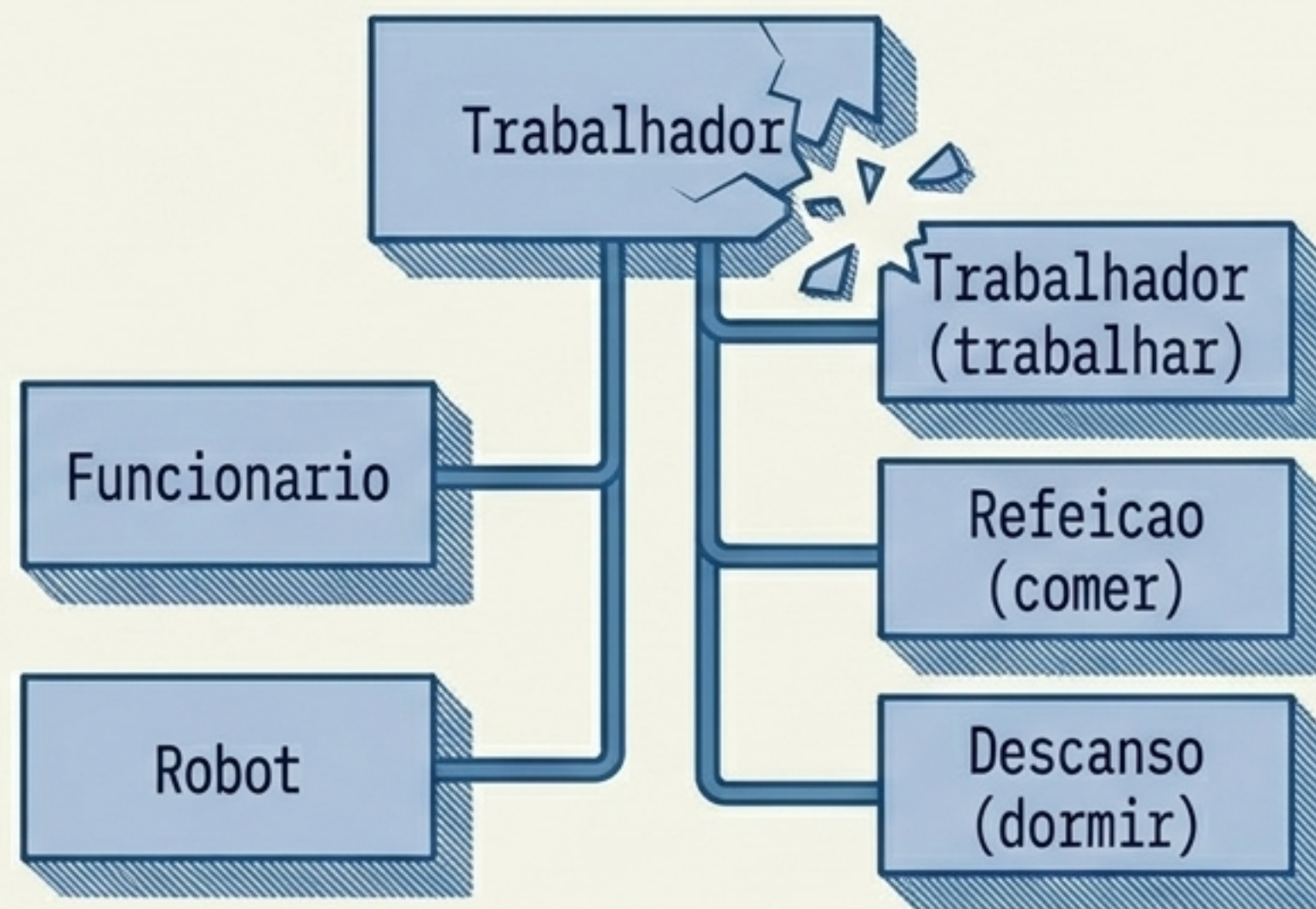
É melhor ter várias interfaces específicas, cirúrgicas e coesas do que uma única interface "faz-tudo" que polui as implementações.

Interfaces Cirúrgicas

Sintoma: Assinaturas Falsas

```
class Robot implements Trabalhador {  
    public void trabalhar() { ... }  
  
    @Override  
    public void comer() {  
        throw new UnsupportedOperationException  
    }  
}
```

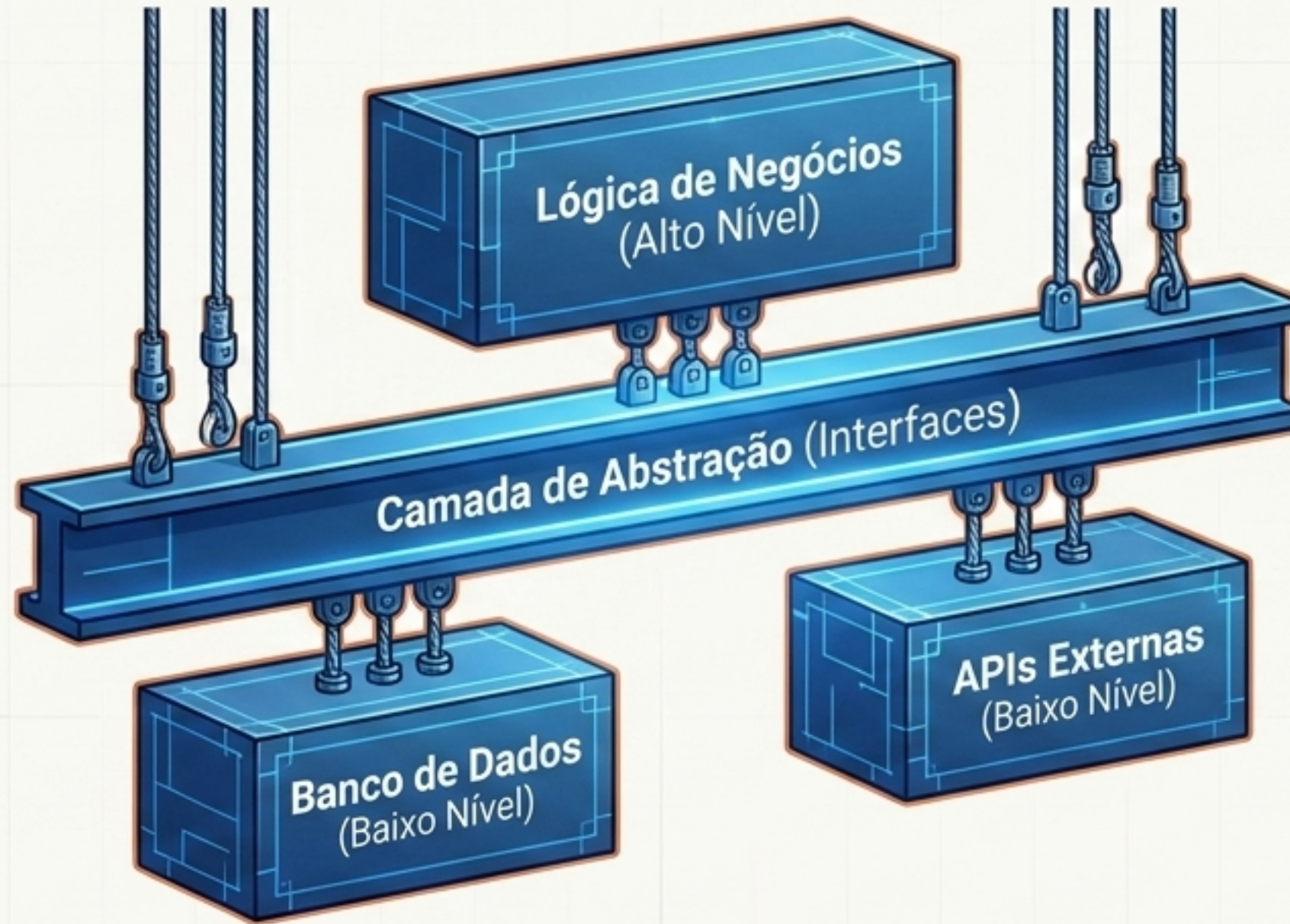
Cura: Interfaces Específicas



Forçar um 'Robô' a implementar um método de 'Comer' polui a base de código. Classes assinam apenas os contratos que podem cumprir.

Dependency Inversion Principle (DIP)

A Regra: Módulos de alto nível não devem depender de módulos de baixo nível. Ambos dependem de abstrações.



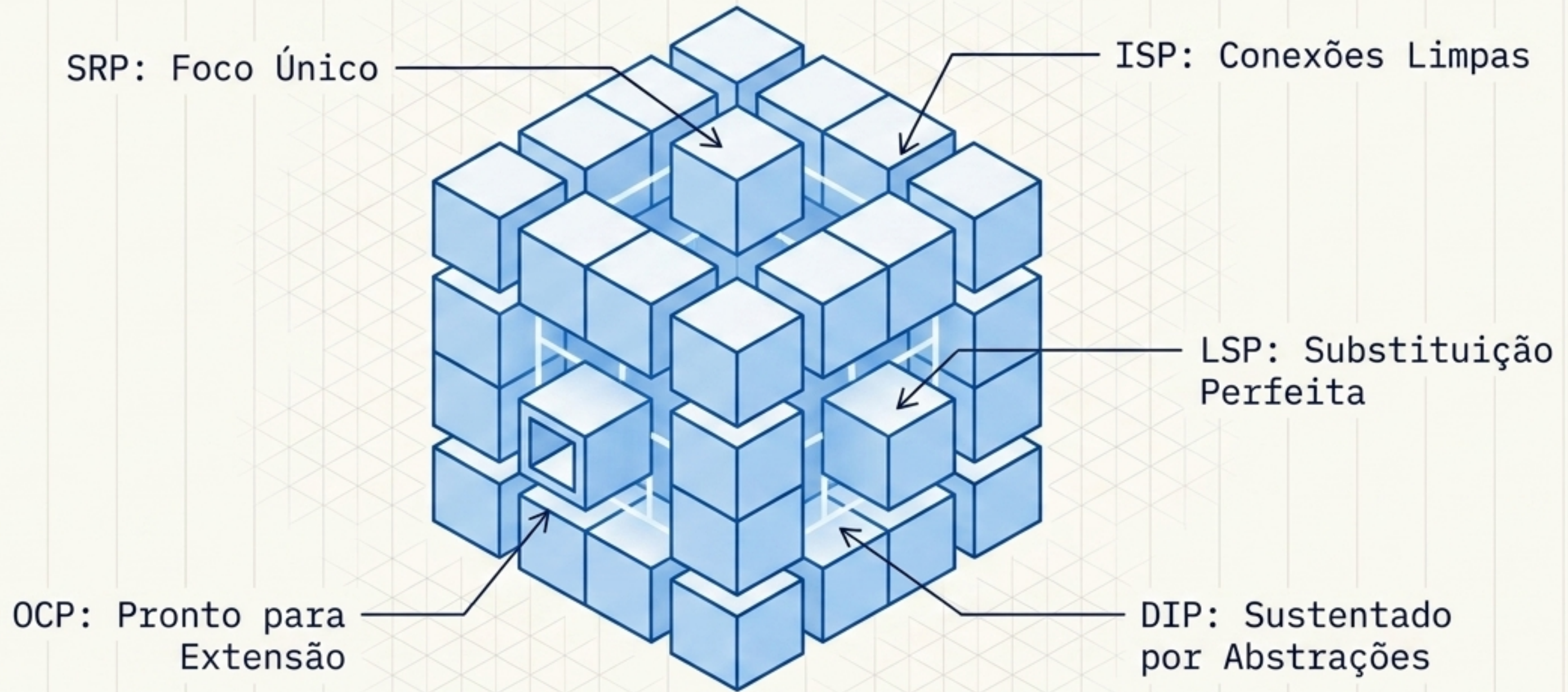
Separa o núcleo do sistema das tecnologias de infraestrutura. Isso permite trocar o banco de dados sem alterar uma vírgula da lógica central.

Arquitetura no Mundo Real

 E-Commerce	SRP isola as lógicas de imposto, estoque e pagamento, permitindo que times paralelos atuem sem conflitos.
 Banking	DIP separa a lógica de negócios da infraestrutura, permitindo migração segura de banco de dados.
 Games	OCP permite plugar novos motores de física ou áudio sem quebrar o motor principal do jogo.
 Web Apps	ISP garante que diferentes microsserviços dependam apenas das interfaces que realmente consomem.
 APIs REST	LSP garante que novos payloads de pagamento possam substituir os base sem alterar o comportamento esperado.

SOLID é a infraestrutura invisível que sustenta a estabilidade das compa tuais das maiores opera  es de software atuais.

Harmonia Arquitetural



Os princípios não existem no vácuo. SRP gera OCP. ISP garante LSP. Juntos, eles formam a engenharia de um software resiliente.